SHARENET – CAMPUS SHARING ECONOMY PLATFORM

CS23531 – WEB PROGRAMMING
MINI PROJECT REPORT

Submitted by

HARISH S – 2116241001081

Of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**SHARENET – CAMPUS SHARING ECONOMY PLATFORM**" is the bonafide work of **HARISH S (2116241001081)** who carried out the work under my supervision.

SIGNATURE
Dr. P. Neelaveni
HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

SIGNATURE
Mrs. Janani M
COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

This project is submitted for CS23531 – Web Programming held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

ShareNet is a student-focused online marketplace developed using React.js, HTML, CSS, JavaScript, Node.js and Express.js. It connects college students and encourages resource sharing within the campus community. Students can buy, sell, rent, exchange and give away useful items such as textbooks, calculators, electronic devices, stationery and other academic resources. The platform also provides a Lost and Found section, a Wanted Board for requesting items, real-time messaging between students, student authentication using college credentials, and user profiles with a reputation score. The application follows a client-server architecture in which a React single-page frontend communicates with an Express REST API, and all users, listings, messages and reviews are stored in a database. The main objective of ShareNet is to reduce student expenses, improve access to essential resources, minimise waste through reuse and build a connected, trustworthy campus community. In simple words, ShareNet is like OLX exclusively for college students, with additional features for renting, giving away, finding lost items and sharing resources. Tagline: Share More. Spend Less. Connect Better.

TABLE OF CONTENTS

S.No	Section Title
1	1.1 Introduction
2	1.2 Purpose
3	1.3 Scope of the Project
4	1.4 Software Requirements Specification
5	2.1 Use Case Diagram
6	2.2 Entity Relationship Diagram
7	2.3 Data Flow Diagram
8	3. Module Description
9	4.1 User Interface and Workflow
10	4.2 Data Design
11	4.3 Selected Source Code
12	4.4 Testing Scenarios
13	5. Conclusion
14	6. References

1. PROJECT OVERVIEW

1.1 INTRODUCTION

Students spend a large amount of money every semester on textbooks, calculators, lab equipment and other academic items that are often used only for a short time and then left unused. Existing general marketplaces are open to everyone, which makes it hard to find items on campus and to trust the person on the other side. ShareNet provides a centralised web portal exclusively for students of a college, where they can list items to sell, rent, exchange or give away, discover resources posted by classmates, report lost and found items and chat directly to arrange a hand-over. The application is implemented with a React.js frontend and a Node.js and Express.js backend.

1.2 PURPOSE

- Reduce student expenses by making second-hand and rented resources easy to find.
- Improve access to essential academic resources such as textbooks, calculators and devices.
- Minimise waste by encouraging reuse and free giveaways within the campus.
- Provide a trusted space through student authentication, profiles and reputation.
- Support quick coordination between students through real-time chat.

1.3 SCOPE OF THE PROJECT

- Buy and Sell Marketplace with category-wise listings and keyword search.
- Item Rental System for borrowing resources for a fixed period.
- Free Giveaways and exchange of items between students.
- Lost and Found board to report and recover lost belongings on campus.
- Wanted Board where students can request items they need.
- Real-time chat, student authentication, user profiles and reputation.

Limitations: ShareNet does not process online payments; price, rent and hand-over are settled directly between students. The platform is intended for a single campus community, moderation of listings is basic, and the reputation score depends on the ratings that students choose to give. Native mobile applications and push notifications are not part of the current version.

1.4 SOFTWARE REQUIREMENTS SPECIFICATION

Architecture: client-server web application. The React frontend sends HTTP requests to the Express.js REST API, which validates the student's token, applies the business logic and reads or writes the database. Real-time chat messages are exchanged over a WebSocket connection, and the frontend updates the page as soon as new data arrives.

Hardware requirements

- Processor: dual-core or higher (suggested)
- RAM: 8 GB recommended to run Node.js, the database server, a browser and an IDE together
- Display: standard laptop or desktop

Software requirements

- Windows, macOS or Linux with a modern web browser (Chrome, Edge or Firefox)
- Node.js 20 LTS with the npm package manager

- React.js frontend with HTML, CSS and JavaScript (ES6)
- Express.js backend with JWT authentication, bcrypt password hashing and Socket.IO for chat
- MongoDB 7.x database accessed through the Mongoose library
- Visual Studio Code or an equivalent IDE, with Postman for API testing

Functional Requirements

- Marketplace: create, edit and delete listings to sell, rent, exchange or give away items.
- Search: filter listings by keyword, category, listing type and price.
- Lost and Found / Wanted Board: post and browse lost items, found items and item requests.
- Chat: real-time one-to-one messaging between students about a listing.
- Authentication: register and log in using college credentials; only verified students can post.
- Profiles: view a student's listings and reputation rating, and rate users after a transaction.

Non-functional Requirements

- Simple, responsive interface that works on laptops and mobile browsers.
- Fast page loads and instant chat delivery through a single-page React frontend.
- Secure storage of passwords using hashing and protected API routes using tokens.
- Clear modular code structure that is easy to extend.

2. SYSTEM FLOW DIAGRAMS

2.1 USE CASE DIAGRAM

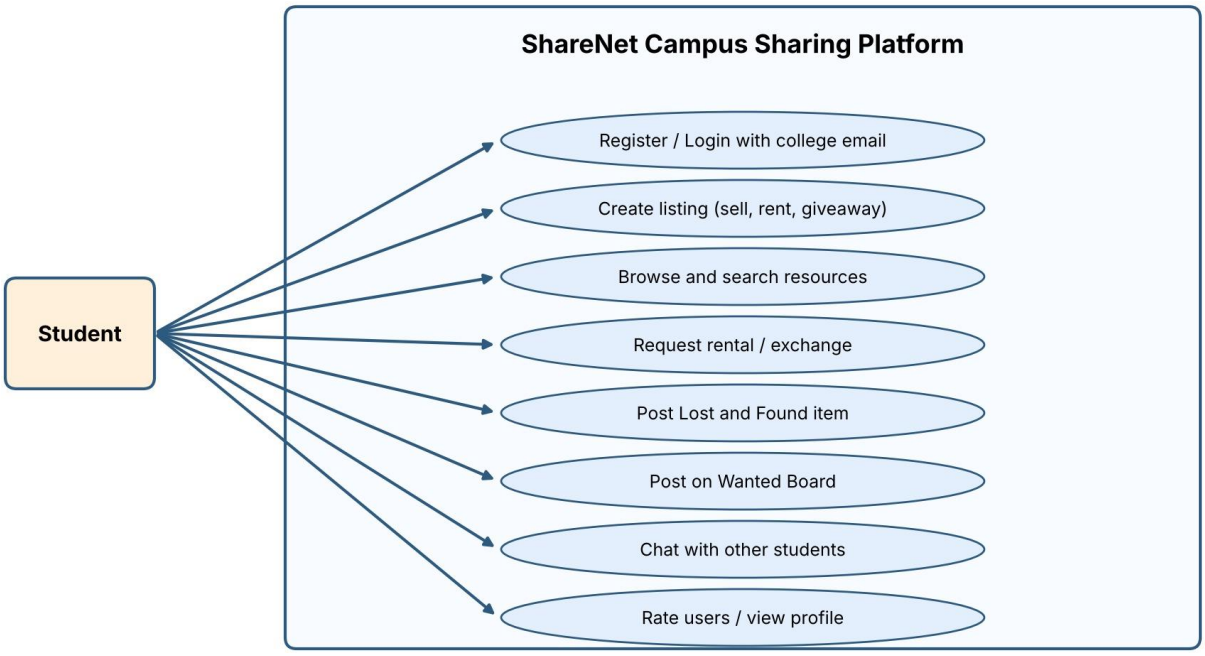


Figure 2.1: Student use cases

2.2 ENTITY RELATIONSHIP DIAGRAM

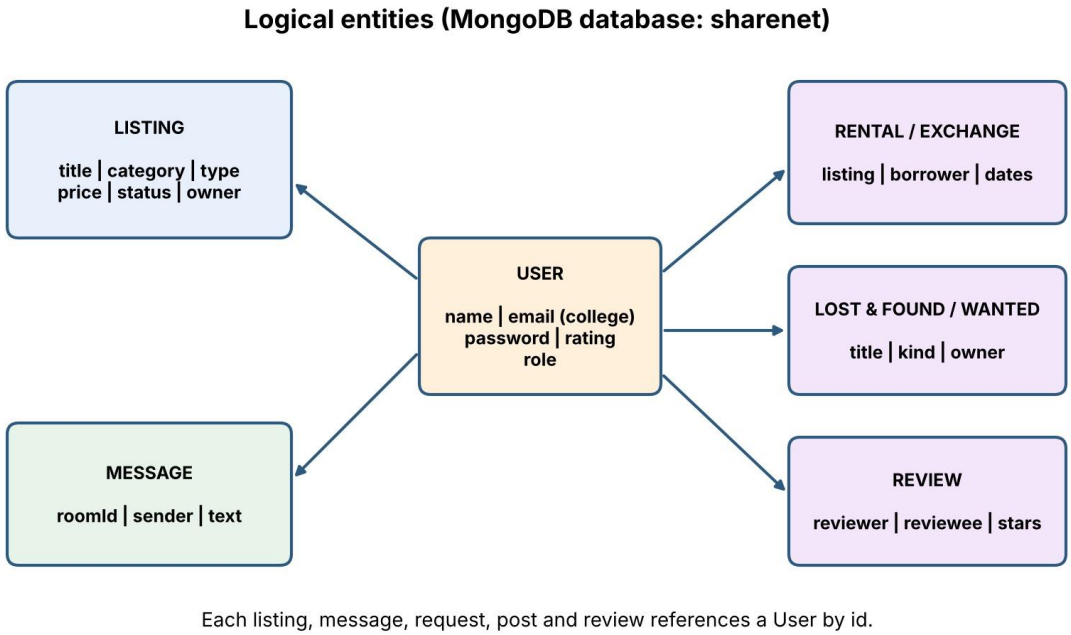


Figure 2.2: Main ShareNet entities stored in the MongoDB database

2.3 DATA FLOW DIAGRAM

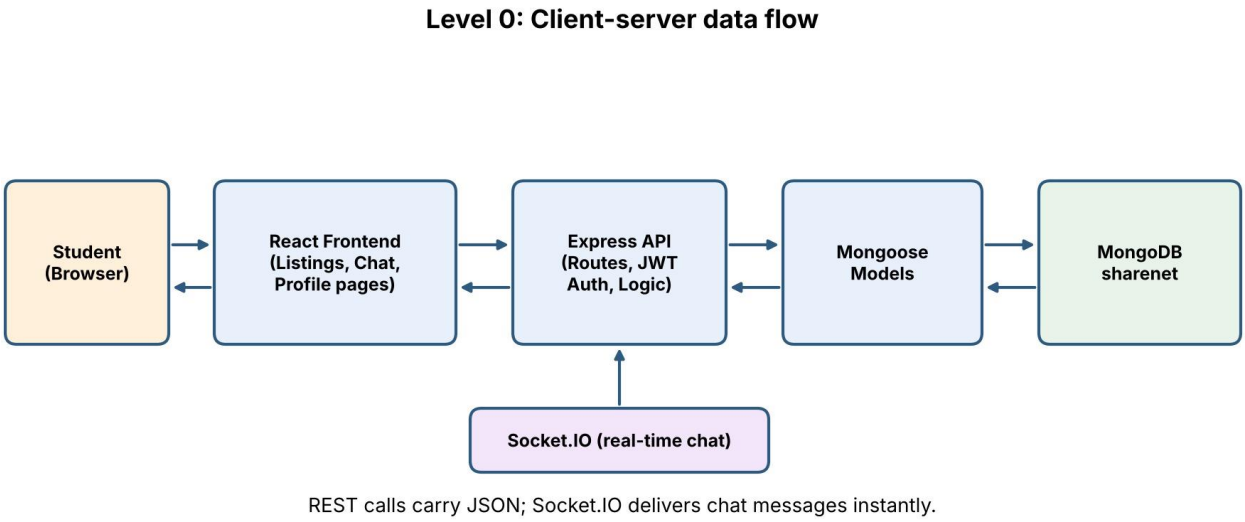


Figure 2.3: Browser, React frontend, Express API and database data flow

3. MODULE DESCRIPTION

- Authentication Module: student registration and login with college credentials, password hashing and token-based access.
- Marketplace Module: create, edit, search and delete listings to buy, sell or exchange items.
- Rental Module: rent listing details, rental requests, dates and return status.
- Giveaway Module: free items offered to the campus community on a first-come basis.
- Lost and Found Module: report lost or found items and browse recent reports.
- Wanted Board Module: post requests for items that students need and respond to them.
- Chat Module: real-time messaging between buyer and owner using Socket.IO.
- Profile and Reputation Module: user profile, listing history, ratings and reviews.

4. IMPLEMENTATION

4.1 USER INTERFACE AND WORKFLOW

Step 1: Install Node.js and MongoDB, then set the database URL and secret key in the .env file. Step 2: Run npm install and npm start in the server folder, then npm install and npm start in the client folder and open http://localhost:3000/. Step 3: Register with a college email and log in to reach the marketplace. Step 4: Create a listing and choose Sell, Rent, Exchange or Giveaway. Step 5: Search and filter listings, open an item and start a chat with the owner. Step 6: Use Lost and Found and the Wanted Board, and review profiles and ratings after a transaction.

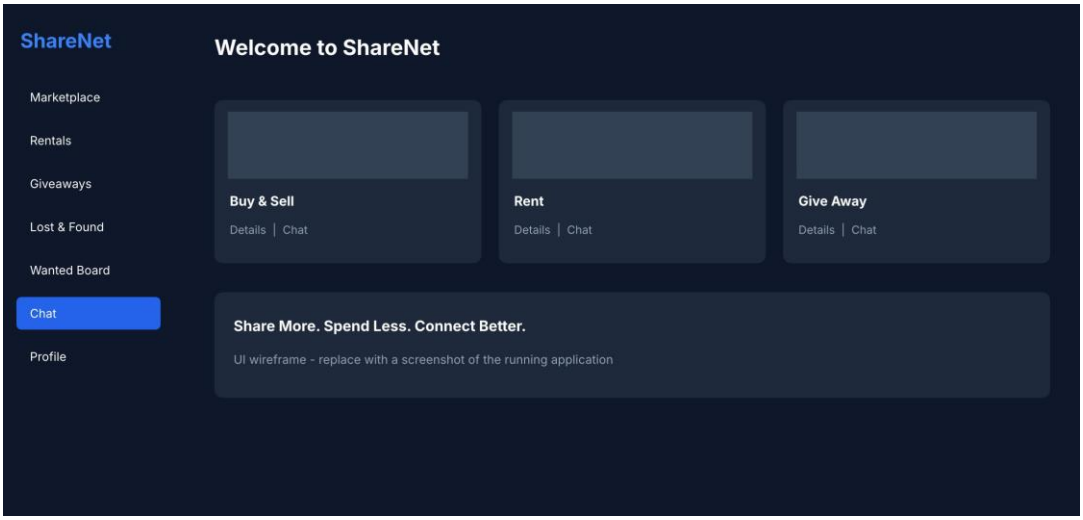


Figure 4.1: Landing page

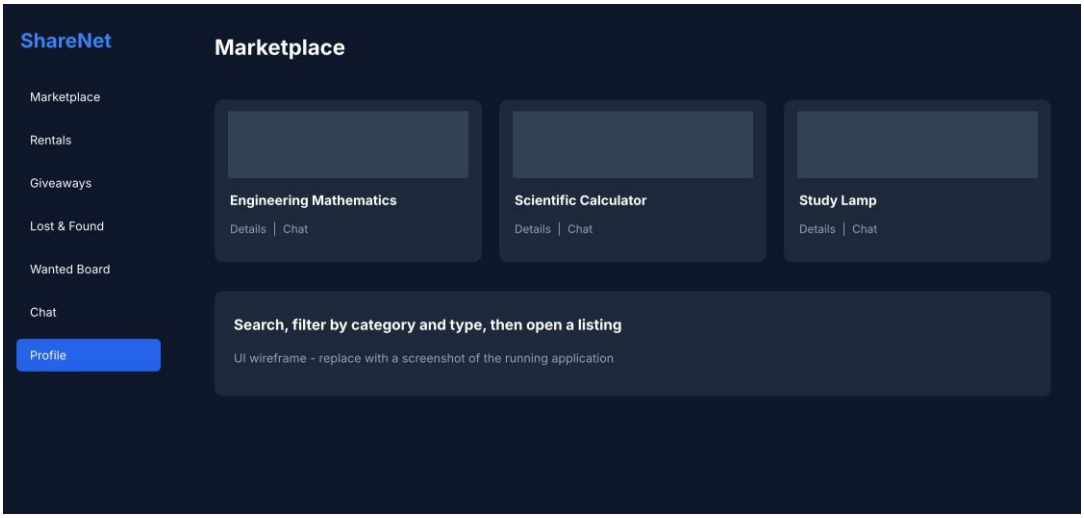


Figure 4.2: Marketplace with searchable listings

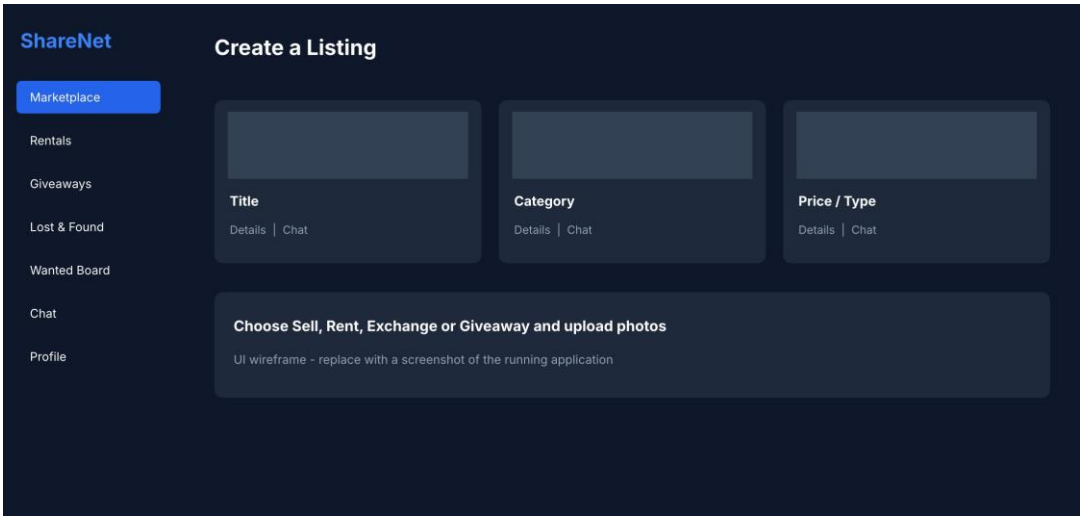


Figure 4.3: Create Listing form for sell, rent, exchange and giveaway items

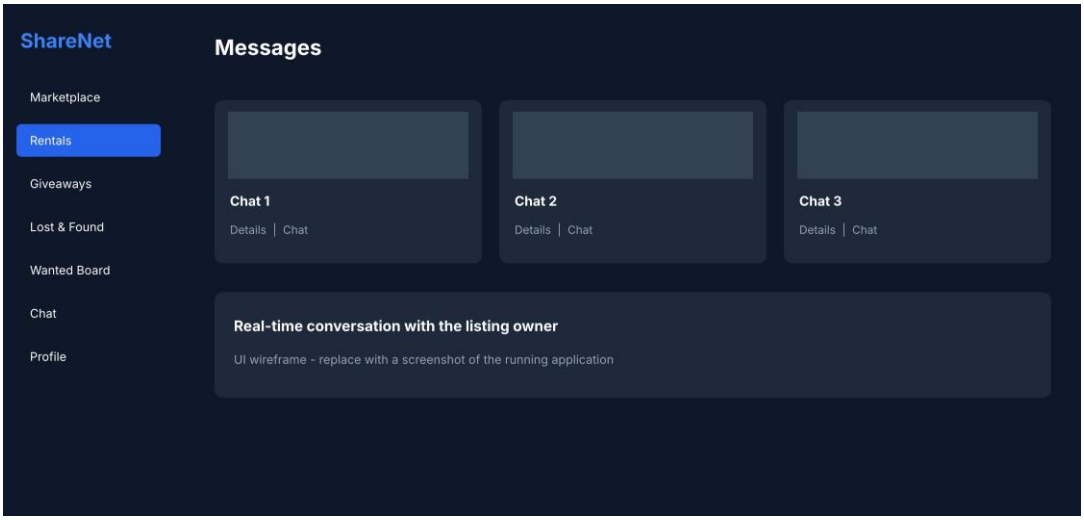


Figure 4.4: Real-time chat between students

4.2 DATA DESIGN

The application stores its data in a MongoDB database named sharenet, which holds the collections users, listings, rentals, messages, lostfound, wanted and reviews. A listing has a title, description, category, type (sell, rent, exchange or giveaway), price, status (available, rented or closed) and an owner that references a user. Each user stores the name, college email, hashed password and reputation rating.

Messages are saved with a room id, the sender and the text so that earlier conversations can be reloaded. Rentals link a listing with the borrower, the start and end dates and the return status, while reviews link a reviewer to the student being rated.

Formula: Reputation = round(sum of review ratings / number of reviews, 1). When a student has no reviews, the profile shows 0 to avoid division by zero. Listings of type giveaway always have price 0.

4.3 SELECTED SOURCE CODE

1. Listing model (models/Listing.js)

```
const mongoose = require("mongoose");

const listingSchema = new mongoose.Schema({
  title: { type: String, required: true },
  category: String,   // Books | Electronics | Stationery | Other
  type: { type: String, enum: ["sell", "rent", "exchange", "giveaway"] },
  price: { type: Number, default: 0 },
  status: { type: String, default: "available" },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: "User" }
}, { timestamps: true });

module.exports = mongoose.model("Listing", listingSchema);
```

2. Authentication middleware (middleware/auth.js)

```
const jwt = require("jsonwebtoken");

module.exports = function auth(req, res, next) {
  const token = (req.headers.authorization || "").split(" ")[1];
  if (!token) return res.status(401).json({ message: "Login required" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ message: "Invalid token" });
  }
};
```

3. Listing routes (routes/listings.js)

```
router.get("/", async (req, res) => {
  const { q, type, category } = req.query;
  const filter = { status: "available" };
  if (type) filter.type = type;
  if (category) filter.category = category;
  if (q) filter.title = new RegExp(q, "i");  // case-insensitive search
  res.json(await Listing.find(filter).sort({ createdAt: -1 }));
});

router.post("/", auth, async (req, res) => {
  const listing = await Listing.create({ ...req.body, owner: req.user.id });
  res.status(201).json(listing);
});
```

4. Student registration (controllers/authController.js)

```
exports.register = async (req, res) => {
  const { name, email, password } = req.body;
  if (!email.endsWith("@" + process.env.COLLEGE_DOMAIN)) {
    return res.status(400).json({ message: "Use your college email" });
  }
  const hash = await bcrypt.hash(password, 10);
  const user = await User.create({ name, email, password: hash });
  res.status(201).json({ id: user._id });
};
```

5. Real-time chat (socket.js)

```
io.on("connection", (socket) => {
  socket.on("join", (roomId) => socket.join(roomId));
```

```
  socket.on("message", async ({ roomId, sender, text }) => {
    const msg = await Message.create({ roomId, sender, text });
    io.to(roomId).emit("message", msg);
  });
});
```

6. Listing card component (ListingCard.jsx)

```
function ListingCard({ item }) {
  return (
    <div className="card">
      <h3>{item.title}</h3>
      <span className={"badge " + item.type}>{item.type}</span>
      <p>{item.type === "giveaway" ? "Free" : "Rs. " + item.price}</p>
      <button onClick={() => openChat(item.owner)}>Chat</button>
    </div>
  );
}
```

7. Configuration (.env)

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/sharenet
JWT_SECRET=********
COLLEGE_DOMAIN=college.edu
```

4.4 TESTING SCENARIOS

Test	Action	Expected result
T1	Register with a valid college email	Account is created and user can log in
T2	Register with a non-college email	Registration is rejected with a message
T3	Create a listing as Sell, Rent or Giveaway	Listing appears in the marketplace
T4	Delete own listing	Listing is removed from the marketplace and database
T5	Search with a partial, mixed-case title	Matching listings are shown
T6	Filter by category and listing type	Only matching listings are shown
T7	Open chat on a listing and send a message	Message reaches the owner instantly
T8	Post a Lost and Found item and a Wanted request	Posts appear on the respective boards

These scenarios are intended to be verified manually on the running application, along with API checks in Postman for the authentication, listing and message routes.

5. CONCLUSION

ShareNet provides a practical campus sharing platform where students can buy, sell, rent, exchange and give away resources, report lost and found items, post wanted requests and chat in real time. By combining student

authentication, profiles and reputation with a React frontend and a Node.js and Express.js backend, the project reduces student expenses, improves access to essential resources and promotes reuse within the campus. Future extensions could include online payments, email and push notifications, image moderation tools, a mobile application, location-based filters and automated unit and integration tests.

6. REFERENCES

- React Documentation — https://react.dev/
- Node.js Documentation — https://nodejs.org/en/docs
- Express.js Guide — https://expressjs.com/
- Socket.IO Documentation — https://socket.io/docs/
- MongoDB Manual and Mongoose Docs — https://www.mongodb.com/docs/ , https://mongoosejs.com/docs/
- Project source: ShareNet web application (React client, Express server, models, routes, socket handler)